

DESIGN PROPOSAL

Modernizing Defense Innovation Through

Set-Based Design, Requirements-Driven Semantic Governance,
and Predictive Scoring Metrics

Prepared for:

Defense Acquisition & Innovation Modernization Program

Version 1.0 | March 2026

CONTROLLED DOCUMENT — FOR OFFICIAL USE ONLY

Table of Contents

1. Problem Statement: How Cold War Practices Hinder Modern Defense Innovation

- 1.1 Premature Design Lock-In via Point-Based Design
- 1.2 Bureaucratic Inertia and Inflexibility
- 1.3 Static Requirements and Cognitive Bias in Analysis

2. Set-Based Design: Core Principles and Defense Application

- 2.1 The Four Value-Improving Tenets
- 2.2 Multiobjective Value of Information in SBD

3. Requirements-Driven Semantic Gateway: Architecture and Implementation

- 3.1 Capture and Analysis of Evolving Requirements
- 3.2 Rationalization and Prioritization
- 3.3 Implementation for AI and Knowledge Engineering
- 3.4 Validation and Measurement

4. SHACL and SWRL: Automated Governance Infrastructure

- 4.1 SHACL: Structural and Compliance Governance
- 4.2 SWRL: Behavioral and Inference Governance
- 4.3 SHACL and SWRL in Set-Based Design Governance

5. Predictive Scoring Metrics and AI Agent Reliability

- 5.1 Strict Propriety: The Foundation of Honest Forecasting
- 5.2 The Three Principal Scoring Rules
- 5.3 Scoring Rule Decomposition and Model Calibration

6. Supporting Evolving System Design and Emergent Behaviors

- 6.1 Closed-Loop Requirements Evolution
- 6.2 Ontology Gap Detection and Knowledge Graph Evolution
- 6.3 Set-Based Design as the Outer Control Loop

7. Implementation Roadmap

8. Conclusion

Executive Summary

The United States Department of Defense faces an urgent challenge: Cold War-era acquisition frameworks are ill-suited to the pace, complexity, and adaptability demanded by modern multi-domain operations and the rapid proliferation of artificial intelligence (AI). This proposal presents an integrated, three-pillar modernization architecture that fuses Set-Based Design (SBD), a Requirements-Driven Semantic Gateway (RDSG), and rigorous Predictive Scoring Metrics to transform how defense systems are conceived, specified, governed, and validated.

Traditional Point-Based Design locks programs into premature design commitments, producing cascading Nunn-McCurdy cost breaches and schedule delays. Simultaneously, static requirements documents cannot keep pace with evolving AI behaviors, emergent system properties, and volatile operational contexts. The proposed framework addresses both failure modes by:

- Deliberately delaying design convergence through SBD, preserving flexibility until uncertainty is resolved.
- Encoding requirements as live, machine-enforceable semantic artifacts governed by SHACL constraints and SWRL inference rules.
- Applying strictly proper scoring rules and Multiobjective Value of Information (MOVOI) analysis to evaluate AI agent reliability, quantify epistemic uncertainty, and prioritize resource allocation.

Together, these pillars create a self-reinforcing governance ecosystem capable of supporting complex adaptive systems, GenAI, Agentic AI, and unmanned platform programs—delivering higher-quality capabilities to warfighters at the speed of relevance.

1. Problem Statement: How Cold War Practices Hinder Modern Defense Innovation

The Department of Defense's acquisition system was engineered for an era defined by long development cycles, predictable adversary capabilities, and limited commercial technology relevance. Three structural pathologies inherited from that era actively undermine innovation today:

1.1 Premature Design Lock-In via Point-Based Design

Point-Based Design (PBD), the dominant historical methodology, attempts to converge on a single baseline solution early in the development lifecycle. Because a design decision represents an irrevocable commitment of finite resources—engineering hours, tooling investment, supply chain configuration—early lock-in forecloses the exploration of potentially superior alternatives. As requirements inevitably evolve, programs face a binary choice: absorb expensive design rework and remediation, or field a system that no longer meets the operational need. The result is the chronic pattern of cost overruns, schedule slippage, and capability shortfalls that characterize major defense acquisitions.

1.2 Bureaucratic Inertia and Inflexibility

Cold War acquisition regulations, milestones, and oversight structures were designed for hardware-centric, long-lead programs. They struggle to accommodate iterative software development, commercial AI model adoption, and agile hardware-software co-design. Program offices are incentivized to preserve approved baselines rather than incorporate emerging technologies mid-cycle, creating a structural lag between fielded capabilities and the state of the art.

1.3 Static Requirements and Cognitive Bias in Analysis

Traditional requirements documents are authored once, approved through a rigid gated process, and subsequently treated as fixed constraints. This approach is vulnerable to attribute substitution—a cognitive bias in which complex, intangible system qualities (operational sustainability, user cognitive load, resilience under adversarial conditions) are unconsciously replaced by simpler, measurable proxies during both requirements elicitation and design evaluation. The substitution is invisible to reviewers and produces architectures that satisfy documented metrics while failing actual operational objectives.

Furthermore, static documents cannot represent the emergent behaviors, inter-domain contextual dependencies, or volatile scenario-specific knowledge that characterize modern AI-enabled systems. As GenAI and Agentic AI systems are integrated into defense platforms, the gap between documented requirements and actual system behavior widens unless a dynamic, machine-readable requirements substrate is maintained.

Cold War Pathology	Consequence for Modern Programs
Point-Based Design (PBD)	Premature lock-in → costly rework, Nunn-McCurdy breaches
Bureaucratic milestone structures	Inability to adopt commercial AI and agile methodologies

Cold War Pathology	Consequence for Modern Programs
Static requirements documents	Attribute substitution, emergent behavior blindspots, requirements drift
Shrinking defense industrial base	Reduced competition, concentrated single-point-of-failure supply chains

Table 1. Cold War acquisition pathologies and their modern consequences.

2. Set-Based Design: Core Principles and Defense Application

Set-Based Design (SBD) is a concurrent, iterative engineering methodology that addresses PBD's structural failure modes by maintaining and systematically maturing a tradespace of design alternatives, rather than prematurely converging on a single solution. Originally developed in the Toyota Production System and subsequently adapted for complex engineered systems, SBD is now recognized by the Office of the Secretary of Defense as a key enabler of resilient acquisition programs.

2.1 The Four Value-Improving Tenets

Tenet 1 — Deliberate Decision Delay

SBD purposely defers design selection decisions until primary sources of epistemic uncertainty—technology readiness, requirement feasibility, budget constraints, threat evolution—are sufficiently resolved through analysis, simulation, and prototyping. This is not indecision; it is a disciplined strategy to conserve optionality. By delaying the irrevocable commitment of resources, programs avoid the performance, cost, and schedule risks associated with premature selection of infeasible or suboptimal designs.

In the context of defense acquisition, deliberate decision delay directly prevents Nunn-McCurdy cost breaches. Nunn-McCurdy is triggered when unit costs exceed the approved baseline by 15% (significant breach) or 25% (critical breach)—thresholds routinely crossed when programs are forced into expensive redesign after early lock-in. By holding design options open, SBD eliminates the principal cause of these overruns.

Tenet 2 — Robust Alternative Development

Rather than focusing analytical resources on refining a single design, SBD concurrently develops and evaluates a wide tradespace of system and sub-system alternatives grouped into feasibility "sets." Multiobjective decision analysis tools—including Pareto optimality analysis, response surface methods, and model-based systems engineering (MBSE) simulations—evaluate alternatives across cost, schedule, performance, and risk dimensions simultaneously. This breadth dramatically increases the probability of discovering solutions that are resilient, affordable, and genuinely Pareto-optimal across stakeholder value dimensions.

Tenet 3 — Uncertainty Reduction and Resolution

SBD employs a structured uncertainty quantification and reduction cycle. Multi-resolution (MR) modeling enables engineers to examine design alternatives at varying levels of fidelity—from fast, coarse-grained parametric analyses to high-resolution computational models and hardware-in-the-loop simulations. As models resolve epistemic uncertainty, infeasible or dominated design sets are formally eliminated, narrowing the tradespace through evidence-based convergence rather than schedule pressure or administrative decision.

Tenet 4 — Maintained Design Flexibility

Because multiple viable design options remain open deep into the program lifecycle, SBD provides the structural flexibility to integrate emerging technologies—advanced materials, novel sensor modalities, updated AI models—without triggering a full program reset. This is particularly critical for software-intensive systems and unmanned aerial vehicles (UAVs), where

hardware-software interfaces evolve rapidly, and commercial off-the-shelf AI components may be superseded by improved alternatives within a single program increment.

2.2 Multiobjective Value of Information in SBD

A core quantitative instrument within the SBD framework is the Multiobjective Value of Information (MOVOI) analysis. MOVOI does not reduce uncertainty directly; rather, it is a decision-support methodology that quantifies the expected benefit of acquiring specific information—running a high-resolution simulation, conducting a prototype test, commissioning an independent cost estimate—relative to its implementation cost.

MOVOI provides program managers with three critical capabilities:

- **Decision Change Assessment:** Determining whether acquiring new high-fidelity data will change which design set is selected, thereby preventing resources from being spent on information that cannot alter the outcome.
- **Comparative Analytics:** Ranking multiple candidate analytical activities (simulations, experiments, expert elicitations) by their expected information yield, enabling structured prioritization of the modeling and simulation budget.
- **Emergent Behavior Management:** For complex adaptive systems in which unexpected emergent behaviors are inherently unpredictable, MOVOI identifies which modeling activities are most likely to reveal system-level interactions that could invalidate current design assumptions.

It is important to note that MOVOI, by evaluating imperfect information sources, is highly effective at mitigating and managing uncertainty but cannot eliminate the fundamentally unpredictable nature of complex emergent behaviors. This limitation makes the semantic governance layer described in Section 3 an essential complement.

3. Requirements-Driven Semantic Gateway: Architecture and Implementation

The Requirements-Driven Semantic Gateway (RDSG) transforms requirements from static prose documents into first-class semantic citizens: machine-readable, machine-enforceable, and dynamically linked to the AI systems and design frameworks they govern. This transformation is foundational to managing the volatile growth of scenario-specific knowledge, the evolution of inter-domain contextual dependencies, and the emergence of unpredictable AI behaviors.

3.1 Capture and Analysis of Evolving Requirements

Requirements elicitation in complex adaptive systems is vulnerable to attribute substitution—a well-documented cognitive bias in which abstract, high-value system qualities are unconsciously replaced with simpler, more measurable proxies. The RDSG's capture architecture is specifically designed to counteract this bias.

Five-Stage Capture Pipeline

1. **Elicitation:** Structured interviews, operational scenarios, and Jobs-to-Be-Done (JTBD) analysis capture what progress each stakeholder persona is genuinely trying to make, independent of specific features or technologies.
2. **Classification:** A semantic parser classifies each elicited requirement into formal ontology subclasses: `FunctionalRequirement`, `ComplianceRequirement`, `NonFunctionalRequirement`, `ScenarioRequirement`, or `ControlRequirement`.
3. **Contextualization:** Each requirement is linked to a structured context model mapping `Persona` → `FunctionalJob` → `JobContext` → `DesiredOutcome`. This interpretive layer enables the system to validate whether AI agent actions genuinely advance the persona's objective.
4. **Constraint Binding:** Requirements are linked to SHACL constraint shapes and SWRL inference rules that will enforce them at runtime.
5. **Acceptance Criterion Specification:** Every requirement is associated with at least one testable, executable `AcceptanceCriterion` that serves as a validation gate rather than passive documentation.

Deterministic domain knowledge—stable facts, regulations, taxonomies, and process definitions—is encoded in formal OWL ontologies. Volatile, scenario-specific knowledge—operational context, threat assessments, ephemeral mission parameters—is captured as `ScenarioRequirement` individuals and managed through a vector database architecture that supports rapid semantic retrieval.

3.2 Rationalization and Prioritization

In systems with dozens of stakeholders and hundreds of requirements, prioritization must be driven by stakeholder value rather than political influence or analytical convenience. The RDSG implements Value-Focused Thinking (VFT), which structures the design space around fundamental objectives rather than alternative comparison.

- **MoSCoW Classification:** Requirements are formally classified as Must, Should, Could, or Won't Have via the `req:priority` property, creating an unambiguous prioritization basis for program planning.
- **Conflict Detection:** The `req:dependsOn` and `req:conflictsWith` properties create a live dependency graph. When two active, conflicting requirements are simultaneously triggered, SWRL rule REQ-008 automatically halts AI directive compilation and routes a structured `REQUIREMENT_CONFLICT` record to the governance queue for human domain steward resolution.
- **Version Control and Traceability:** Every requirement individual maintains a full provenance record—author, timestamp, basis document, approval authority—supporting Defense Acquisition System audit requirements.

3.3 Implementation for AI and Knowledge Engineering

The RDSG's implementation architecture is specifically designed to extend governance to GenAI and Agentic AI systems, which require context that is structured, current, and aligned with operational requirements rather than generic semantic similarity.

Requirements-Derived Query Envelopes (RDQEs)

Requirements-Derived Query Envelopes are the RDSG's primary mechanism for **bridging the semantic requirements graph and vector database retrieval**. When an AI agent is invoked, the gateway constructs a Requirement Context Bundle (RCB) from the active persona and job context, then uses that bundle to modify vector retrieval through seven structured mechanisms:

- **Job Embedding Bias:** The standard query embedding vector is blended with the active `FunctionalJob` embedding, biasing retrieval toward information relevant to the specific task being performed.
- **Outcome Keyword Boost:** Chunks containing terminology matching `DesiredOutcome` metric labels receive an automated relevance score bonus, focusing context on operational success criteria.
- **Scenario Pinning:** When a `ScenarioRequirement` is active, chunks tagged with matching `ScenarioContext` IDs bypass general similarity ranking and are retrieved first, delivering exact situational context.
- **Compliance Mandatory Fetch:** Chunks tagged with active `ComplianceRequirement` references—regulations, policies, standards—are unconditionally prepended to the AI context window, ensuring persistent regulatory awareness.
- **Barrier Suppression:** Active `JobBarrier` entities flag chunks containing known terminology inconsistencies for disambiguation before delivery, preventing AI hallucination from conflicting terminology.
- **Persona Access Filter:** Retrieval is scoped to the querying persona's authorized data access level, enforcing information security and need-to-know controls.
- **NFR Retrieval Budget:** Active `NonFunctionalRequirement` performance thresholds govern total chunk count and maximum retrieval latency, guaranteeing fast-path performance in time-critical operational scenarios.

RDQEs transform the vector database from a generic semantic search index into a structured, requirement-driven knowledge substrate. The result is AI context that is not merely semantically relevant but ontologically governed—aligned with formal requirements, bounded by access controls, and enriched with mandatory compliance context.

3.4 Validation and Measurement

The RDSG implements a **four-layer measurement framework** that continuously validates system performance against deterministic requirements.

Measurement Dimension	RDSG Mechanism
Performance	Technical Performance Measures (TPMs) tracked against Acceptance Criterion thresholds; automated alerts on threshold breach.
Value	Value-Focused Thinking metrics mapped to DesiredOutcome individuals; continuous scoring against persona objective fulfillment.
Operational Integrity	SWRL REQ-007 enforces NonFunctionalRequirement SLAs (latency, availability, throughput) as runtime execution ceilings.
Compliance	ComplianceRequirement instances enforce ISO 31000, GDPR, and DoD-specific regulatory frameworks via dynamically injected SHACL shapes.

Table 2. RDSG measurement framework across four validation dimensions.

4. SHACL and SWRL: Automated Governance Infrastructure

SHACL (Shapes Constraint Language) and SWRL (Semantic Web Rule Language) are the RDSG's primary governance enforcement mechanisms. They transform the requirements ontology from a passive knowledge representation into an active, machine-enforceable governance layer that validates AI outputs, resolves conflicts, and maintains audit trails without human intervention.

4.1 SHACL: Structural and Compliance Governance

SHACL operates as the structural integrity layer of the governance framework. It defines the mandatory properties, cardinality constraints, and value ranges that all requirement individuals must satisfy.

Requirements Completeness Validation

The `sh:RequirementCompleteness` shape enforces that every draft requirement individual includes: exactly one associated `Persona`, a declared `Domain`, a `MoSCoW` priority classification, a `lifecycle Status`, and at least one executable `AcceptanceCriterion`. Requirements that fail this shape are blocked from activation, preventing incomplete specifications from propagating into design or AI directive compilation.

Dynamic Output Validation Gates

- When a prompt triggers a `ControlRequirement` linked to a regulation or policy, the RDSG dynamically injects the associated SHACL shape into the active evaluation set. Before an AI response is delivered, the Service Facade validates the response against:
- **Structural Completeness:** JSON Schema and SHACL validation confirm the response contains all required fields and adheres to the declared output structure.
 - **Ontology Consistency:** SPARQL ASK queries verify that no undefined terms, unresolved references, or ontology violations appear in the generated output.
 - **Outcome Metric Coverage:** The response is checked for inclusion of all `DesiredOutcome` metrics specified in the active Requirement Context Bundle.

4.2 SWRL: Behavioral and Inference Governance

While SHACL handles structural validation, SWRL rules govern dynamic, context-sensitive behaviors. Stored in the relational database, SWRL rules fire based on requirement class membership and active user contexts, modifying the Compiled AI Directive (CAD) in real time.

Rule	Trigger Condition	Governance Action
REQ-001	FunctionalRequirement active	Attaches outcome measurement instructions to the AI prompt; links <code>AcceptanceCriterion</code> as validation gates.
REQ-002	ScenarioRequirement matched	Makes scenario-specific <code>AcceptanceCriteria</code> act as executable validation gates before response delivery.

Rule	Trigger Condition	Governance Action
REQ-004	FunctionalRequirement without SHACL/SWRL binding	Flags missing governance mechanism; routes to domain steward for constraint assignment.
REQ-005	JobBarrier terminology inconsistency detected	Forces disambiguation of conflicting terminology before AI directive compilation proceeds.
REQ-007	NonFunctionalRequirement with performance threshold	Attaches latency ceiling and throughput SLA directly to AI execution parameters.
REQ-008	Two active, conflicting requirements triggered simultaneously	Halts directive compilation; logs structured REQUIREMENT_CONFLICT to governance queue for human resolution.

Table 3. Key SWRL governance rules and their behavioral effects.

4.3 SHACL and SWRL in Set-Based Design Governance

The integration of SHACL and SWRL with the SBD framework creates a feedback loop between design space management and requirements governance. As design sets are evaluated and eliminated through the SBD convergence process, associated requirements may be deactivated, modified, or superseded. SHACL shapes validate that requirement modifications maintain structural integrity, while SWRL rules automatically propagate constraint changes to active AI directives—ensuring that design decisions are immediately reflected in the governance layer without manual synchronization.

5. Predictive Scoring Metrics and AI Agent Reliability

Scoring rules provide a mathematically rigorous framework for evaluating the quality of probabilistic forecasts—whether generated by human analysts, statistical models, or AI agents. In the context of defense acquisition, scoring rules serve three functions: validating the reliability of AI systems integrated into design and decision processes; training AI models to produce calibrated uncertainty estimates; and providing objective metrics for comparing competing AI systems or design configurations.

5.1 Strict Propriety: The Foundation of Honest Forecasting

A scoring rule is strictly proper if its expected score is minimized (or reward maximized) only when the forecaster reports probabilities that exactly match their true beliefs about the outcome distribution. Strict propriety is the mathematical guarantee that eliminates the incentive for strategic misreporting.

When an AI agent—or a human analyst—is evaluated using a strictly proper scoring rule, the agent is mathematically incentivized to report honest probability estimates rather than gaming the metric. In defense acquisition contexts where AI forecasts inform major resource commitments, this property is essential for maintaining the integrity of the decision-support infrastructure.

5.2 The Three Principal Scoring Rules

Three scoring rules are particularly relevant to defense AI evaluation, each with distinct mathematical properties and practical trade-offs:

Brier Score

The Brier Score computes the mean squared error between forecast probabilities and binary outcomes. It is bounded between 0 (perfect) and 1 (maximally wrong), making it intuitive for program managers. Its decomposition into uncertainty, resolution, and reliability components provides diagnostic insight: the reliability component directly penalizes miscalibrated forecasts, while the resolution component rewards forecasters who correctly distinguish high- and low-probability events. The Brier Score's squared error structure is relatively forgiving of overconfident forecasts that happen to be directionally correct, which may be appropriate for rapid prototyping evaluations.

Logarithmic Score

The Logarithmic Score penalizes extreme overconfidence severely—assigning a score of negative infinity when a forecast assigns zero probability to an event that occurs. This property makes it ideal for evaluating AI systems in safety-critical applications where confident misclassification is catastrophic. The logarithmic score is equivalent to the cross-entropy loss function widely used in training classification models, providing a direct connection between evaluation and model training.

Spherical Score

The Spherical Score normalizes the probability vector to the unit hypersphere before scoring, making it invariant to the scale of the probability distribution. This property is particularly useful when comparing AI models that produce systematically different probability magnitudes, or when evaluating models across tasks with different base rates.

Rule	Formula Type	Overconfidence Penalty	Best Application
Brier	Squared error	Moderate (quadratic)	Program manager dashboards, design trade studies
Logarithmic	Log probability	Extreme (infinite at zero)	Safety-critical AI, compliance forecasting
Spherical	Normalized inner product	Moderate (bounded)	Cross-model comparison, multi-task evaluation

Table 4. Comparison of the three principal strictly proper scoring rules.

5.3 Scoring Rule Decomposition and Model Calibration

The expected value of a proper scoring rule decomposes into three interpretable components that provide diagnostic insight into forecast quality beyond a single aggregate metric:

- **Uncertainty:** The baseline expected score of a forecaster that always predicts the marginal event frequency. This establishes the minimum achievable score for a naive model and quantifies the inherent difficulty of the prediction task.
- **Resolution:** The degree to which the forecaster's predictions distinguish between future events—how different the predicted probabilities are when events do versus do not occur. High resolution indicates the model is genuinely discriminating between outcomes.
- **Reliability (Calibration):** The component that penalizes forecasts where predicted probabilities systematically diverge from observed frequencies. A perfectly calibrated model earns zero reliability penalty; miscalibrated AI systems are directly identified and quantified.

Training AI models using strictly proper scoring rules as loss functions teaches the model to accurately represent its own uncertainty—to know when it does not know. This calibration property is critical for defense AI applications where decision-makers must correctly interpret confidence levels when allocating resources or accepting mission risk.

6. Supporting Evolving System Design and Emergent Behaviors

Complex adaptive systems—including multi-domain defense platforms, AI-enabled autonomous vehicles, and large-scale command-and-control architectures—exhibit emergent behaviors: system-level properties that arise from interactions between components and cannot be observed or predicted by decomposing the system into its parts. Managing these emergent behaviors requires an architecture that couples iterative design frameworks with automated feedback loops and continuous knowledge refinement.

6.1 Closed-Loop Requirements Evolution

The RDSG implements a closed-loop feedback architecture that treats emergent AI behaviors not as anomalies to be suppressed but as information signals that drive continuous requirements refinement:

6. Failure Detection: If an AI agent's response continuously fails an `AcceptanceCriterion`, each failure is logged as a `RequirementsFinding` individual with provenance metadata.
7. Threshold-Based Escalation: When a `RequirementsFinding` accumulates beyond a configurable frequency threshold, the system automatically triggers a `RequirementsReview`, notifying domain stewards.
8. Calibration or Capability Decision: Domain stewards review accumulated findings and determine whether the `AcceptanceCriterion` itself requires calibration (the requirement was specified incorrectly) or the AI capability requires improvement (the requirement is correct but unmet).
9. Ontology and Requirements Update: Approved changes are propagated through the ontology, SHACL shapes, and SWRL rules, and versioned with full audit provenance.

6.2 Ontology Gap Detection and Knowledge Graph Evolution

As AI agents and system operators encounter novel contexts not anticipated during initial knowledge engineering, they will generate terms and concepts that fail ontology resolution. The RDSG handles these gaps through a structured detection and remediation process:

- Gap Logging: Unresolved terms are captured as `OntologyGap` individuals, tagged with the originating persona, scenario context, frequency of occurrence, and operational priority.
- Aggregation and Prioritization: `OntologyGap` individuals are aggregated and ranked by a composite priority score combining frequency, persona seniority, and operational criticality.
- Governed Extension: High-priority gaps are routed to domain knowledge engineers for formalization and incorporation into the ontology through the same change control process as requirement modifications—ensuring that knowledge graph growth is governed, auditable, and reversible.

6.3 Set-Based Design as the Outer Control Loop

The SBD tradespace management process provides the outer control loop within which the RDSG's closed-loop requirements evolution operates. As emergent behaviors reveal new

requirements or invalidate existing ones, SBD's maintained flexibility allows design sets to be updated without triggering a full program rebaseline. The MOVOL framework guides resource allocation toward the testing and simulation activities most likely to resolve the newly identified uncertainties—creating a continuous cycle of uncertainty reduction, requirements refinement, and design convergence that adapts to the actual complexity of the system rather than a predetermined schedule.

7. Implementation Roadmap

The following phased implementation roadmap structures the transition from the current-state point-based, static-requirements acquisition to the fully integrated SBD-RDSG-Scoring framework. Each phase is designed to deliver standalone value while establishing the technical foundation for subsequent phases.

Phase	Timeline	Key Deliverables
Phase 1 Foundation	Months 1–6	Ontology architecture design; initial SHACL/SWRL rule set for top-priority requirement classes; pilot SBD tradespace framework for one active program; baseline scoring rule evaluation of existing AI components.
Phase 2 Integration	Months 7–18	Full RDSG deployment, including RDQE engine; integration with program MBSE toolchain; MOVOI analysis framework for modeling and simulation budget prioritization; scoring rule training pipeline for AI forecasting components.
Phase 3 Governance Maturation	Months 19–30	Closed-loop requirements evolution with automated RequirementsReview triggers; OntologyGap detection and governed knowledge graph extension; cross-program SBD tradespace federation; full AI reliability dashboard.
Phase 4 Enterprise Scale	Months 31–48	Multi-program RDSG federation with shared ontology governance; enterprise MOVOI decision-support capability; standardized AI reliability certification using scoring rule benchmarks; integration with Defense Acquisition System milestone reporting.

Table 5. Phased implementation roadmap.

8. Conclusion

The modernization of defense acquisition is not a single-domain problem. It requires simultaneous reform of engineering methodology, requirements governance, and analytical rigor—pursued in an integrated fashion rather than as independent initiatives.

Set-Based Design addresses the structural failure mode of premature design commitment by maintaining and systematically maturing a tradespace of alternatives until epistemic uncertainty is sufficiently resolved. The Requirements-Driven Semantic Gateway addresses the structural failure mode of static, cognitively biased requirements by transforming requirements into live semantic artifacts that actively govern AI behavior, detect emergent behavioral anomalies, and drive continuous knowledge refinement. Predictive Scoring Metrics address the structural failure mode of unvalidated AI reliability by providing mathematically rigorous, incentive-compatible evaluation frameworks that produce calibrated, honest forecasts.

Together, these three pillars constitute a coherent modernization architecture suited to the complexity, pace, and adversarial character of the emerging operational environment. Programs that adopt this integrated framework will be materially better positioned to:

- Avoid Nunn-McCurdy breaches through disciplined design space management and delayed commitment.
- Govern AI-enabled systems against formally specified, machine-enforceable requirements rather than prose documentation.
- Quantify and continuously improve the reliability of AI forecasting and decision-support components.
- Adapt rapidly to evolving threats, requirements, and technologies without triggering costly program rebaselines.
- Field resilient capabilities to warfighters at the speed of relevance rather than the speed of bureaucratic process.

Modernizing defense innovation is ultimately a question of epistemic discipline: knowing what we know, knowing what we do not know, investing in the right information to resolve uncertainty, and building governance structures that preserve and enforce that knowledge across the full system lifecycle. This proposal provides a principled, implementable framework for achieving that discipline at enterprise scale.